

Day 64

* Python Data Base Communication (PDBC) --- Oracle, My, SQL

Index

- => Limitations of files.
- => Advantages of Python Data Base Communication (PDBC)
- => Setup of Python and database software --- (Python, software, Database soft and module).
- => Importance of third party modules.
- => Installation of third party modules by using Pip tool and IDEs.
- => Steps for developing Python Data Base Communication (PDBC) Applications.
- => Programming examples.
- => Types of Queries.
 - a) DDL Queries
 - b) DML Queries
 - c) DRL Queries.
- => Importance of DDL Queries and their type
 - a) create
 - b) Alter with add and modify
 - c) drop
- => Programming examples (Executing DDL statements through Python program)
- => Importance of DML Queries and their type.
 - a) Insert
 - b) delete
 - c) update.
- => Programming examples (Executing DML statements through Python program)
- => Importance of DRL Queries and their type
 - a) select
- => Programming examples (Executing DRL statements through Python program).
- => meta Data
- => Implementation of meta Data
- => Programming examples

* PDBC).

→ Even we Achieved the Data Persistency by using files, files has the following Limitations

- 1) Files of any language does not contain security bcoz files are unable to provide security in the form of username and password.
- 2) Files are unable to store large amount of data.
- 3) Files are diffusing from one OS to ~~any~~ another OS (files are OS dependent)
- 4) Querying and processing the data from files is very complex bcoz of file data is organized w/o indices and identifying the indices is very complex.
- 5) files does not contain column Names (except CSV Files) and complex to process data.

→ To overcome the limitation of files and to Achieve the Data persistency, we must use the concept of any RDBMS Database software (Oracle, MySQL, MongoDB, DB2, SQL ^{server}, PostgreSQL). etc.

- 1) All RDBMS Database softwares provides security bcoz RDBMS Database software considers user names and password.
- 2) All RDBMS Database software stores large amount of data compared to files
- 3) All RDBMS Database software is same on all types of OSes (OS independent)
- 4) Querying and processing the data from All RDBMS Database softwares is very simple bcoz data of All RDBMS Database software organized records ^{in the} form of

table with Column Names.

5) The data present in any RDBMS Database software is organized in the form of tables with Column Names makes the processing easy.

→ if python program want to communicate with any RDBMS database software then we must use a predefined module and such pre-defined module does not exist in python software.

→ some third party software vendors (Ex: "Anthony Tuininga") developed a module for python programmer to communicate with RDBMS database software and placed in github and third party software modules must be installed.

→ to install any third party software modules in python, we use a tool called "pip" and it is present in C:\users\KVR\AppData\Local\programs\python\script folder

syntax : pip install Module Name (at any windows command prompt)

→ if python program want to communicate with oracle Database, then we must install cx_oracle module OR oracledb module.

examples) pip install cx_oracle (upto python 3.10)
pip install oracledb (python 3.11 and higher)

→ if python program want to communicate with mysql database, then we must install mysql-connector or mysql-connector - python module

examples) pip install mysql-connector
pip install mysql-connector } at any windows command prompt.

* Steps for developing Python Data Base Communication (PDBC)

→ To develop Python Data Base Communication (PDBC) Applications, we need the following prerequisites.

- 1) install python software
- 2) install Data Base software (e.g, oracle, mysql, sqlite, etc.).
- 3) install corresponding Data Base module for Python programmers. (upto python 3.10 version, Install ~~oracle~~ cx_oracle module).

→ After installing the above software, we develop the Application, which establish the communication python software and Database software. To doing this we need to learn the following the steps.

step ①:- import Appropriate Data Base module and other modules if required.

step ②:- Every python program must obtain the connection

step ③:- Every python program must create an object of cursor.

step ④:- Every python must design the query, place the query in cursor object and send for execute in database

step ⑤:- Every python program must process the result of the query

step ⑥:- Every python program I recommend to close the connection from database software.

==

* Communication Between Python Programme and Oracle :-

→ if python program want to communicate with any RDBMS software like oracle then we follow certain step:

- 1) import cx_oracle module or ^{version x other module} oracledb depends on python
- 2) Every python program must get connection ^{if required} from Database software.
- 3) Every python program must create an object cursor
- 4) Every python program must design the Query, place the Query in cursor object and execute.
- 5) Every python program must process the result of Query which is coming from Database software through the cursor

step).

- 1). `import cx_oracle` or `oracledb` depends on python version and other modules if required.

→ When Python programmer want to communicate with Oracle Database then Python programmer must import either `cx_oracle` module (upto Python 3.10) or `oracledb` module (from Python 3.11 and higher).

ex). `import cx_oracle as crc`
(OR)
`import oracledb as orc`

- 2) Every python program must obtain the connection from oracle DB.

→ After the importing either cx_oracle or oracledb module, Python program must get the connection from Oracle Database.

→ To get the connection from oracle database, we must use `connect()`, which is present in `cx_oracle` or `oracledb` module.

syntax

```
varname = oracledb.connect ("username/password@DNS/remote db")
```

```
xarname = oracledb.connect ("username/password @ipAddress/serviceID")
```

→ Varname = represent connection object of type <class, oracledb.Connection>
→ Oracledb = represent name of pre-defined third party module
→ connect() = it is one of the pre-defined function used for obtaining the connection from oracle database.

o username = represents username of oracle database.

o password = represents password of oracle database.

o DNS = DNS stands for Domain Naming System/Service.
(Default DNS of every system is localhost).
DNS represents name of the physical machine where database software resides.

o IP Address = - stands for internet protocol Address.

- IP Address represents physical Address of the machine where database software resides.

- The Default IP Address of Every computer is 127.0.0.1 (Loop Back Address).

o ServiceID = represents Alias name or Alternative name to original Name.

- To find ServiceID of oracle database, we use the following Query at SQL environment.

```
SQL> select * from global_name;
```

```
output
```

```
-----  
GLOBAL_NAME  
-----
```

Oracle C- - - - - ServiceID.

→ Here "username/password@DNS/service ID" is called connection URL of oracle database.

→ if we write or specify any part of "username/password@DNS/service ID" as wrong then we get exception called "oracledb.DatabaseError".

3) Every python program must create an object of cursor:-

→ The purpose of creating an object of cursor is that "To carry the Query from python program to oracle Database and brings the result from oracle Database and handover to python program."

⇒ To create an object of cursor, we use a pre-defined function called cursor() which is present in connection object.

syntax:

Varname = Connobj.cursor()

Here Varname is an object of <class, oracle.db.cursor>

program

Q. write a python program which demonstrates obtaining the connection from oracle database.

ex 1

import oracledb as orc # step 1.

try:

conn = orc.connect("system/tiger@localhost/orcl") # step 2

print("Python program obtains connection from oracle db")

print("Type of conn=", type(conn))

except orc.DatabaseError as db:

print("Invalid username/password; login denied")

ex 2

we don't print else: me likhna Hai.

Q. write a program which will demonstrate how to create an object of cursor.

ex 1. import cx_oracle # step 1

try:

con = cx_oracle.connect("system/manager@localhost/x") # step 2

print("\n Python program obtains connection from oracle db")

cur = con.cursor() # step 3

print("\n Type of cur=", type(cur))

print("Python program creates cursor object")

except cx_oracle.DatabaseError as db:

print("problem in oracle DB: db")

4) Every python must design the Query, place the Query in cursor object and send for execute in Database.

→ A Query is a request/question to the Data Base software for performing any Database operation.

→ To execute the Query from python program in Oracle Database, we use a pre-defined function `execute()`, which is present in cursor object.

syntax) `curobj.execute("Query")`

→ Here Query can either DDL, DML and DRL

Oracle database ← --- Collection of tables ← --- Collection of records.

Employee			
eno	ename	sal	dsg
1	RS	1.4	Author
2	TR	2.3	Sci
3	-	-	-

DDL
⇒ Creating table with col Names ← - physical level
1) create, 2) Alter, 3) drop level

DML
⇒ placing records ← - Concept/Logical level
1) insert 2) delete. 3) update

DRL
⇒ selecting records from table ← - view level
1) select.

* Types of Queries in Database softwares

1) DDL (Data Definition Language)

→ The purpose of DDL (Data Definition Language) Queries is that "to deal with physical level of tables such as Table creation with column names, dropping tables and restructuring columns of table".

→ DDL Queries are classified into 3 types. They are

- 1) create
- 2) Alter
- 3) drop.

1) Create

→ it is used for creating table in Database software.

syntax:

SQL > create table table-name (col1 DB Data Type, col2 DB Data Type - - - - - coln) .

example:

SQL > create table student (sno number(2) primary key, sname varchar2(10) not null, marks number(5,2) not null);

2) Alter - - - add option modify option.

→ This query is used for Altering table structure.

→ In other words, alter is used for modifying the column sizes (modify) and adding new column names (add).

syntax (1):

SQL > alter table table-name modify (existing col-name-1 DB Data Type, - - - existing col-name-n DB Data Type).

example:

SQL > alter table ~~table~~ teacher modify (tno number(3), tsal number(6,2));

syntax (2):

SQL > alter table table-name add (^{new col-name-1 Data Type,} ~~col-name-1 Data Type~~ - - - new col-name-n DB Data Type).

example:

SQL > alter table teacher add (cname varchar2(10) not null);

3) drop

→ This query is used for removing or dropping the table from Database software.

syntax:

SQL > drop table tablename

example

SQL > drop table employee

Program.

Q. write a python program which will demonstrate create an employee table with eno, ename, and sal.

ex(1). ~~def~~ import cx_oracle # step 1.

def table_create():

try:

con = cx_oracle.connect("system/manager@
localhost/Xe") # step 2

cur = con.cursor() # step 3

step 4

ct = "create table employee (eno number(2)

primary key, ename varchar2(10) not
null, mname number(5,2), not null);

~~cur~~

cur.execute(ct)

step 5

print("Employee table created successfully in
oracle database --- verify")

except cx_oracle.DatabaseError as db:

print("problem in oracle DB", db).

main program

tablecreate()

Q. write a python program which will change the column size of
(empno, empname) of employee table

→ total ex(1) की संख्या, सही है।

Bas def tableColSize():

step 1

aq = "alter table employee modify (eno number(3), ename
varchar2(15))"

~~cur~~

==

Alter table
col size

Q. Write a python program which will add new column name called company name for employee table.

11/8/2018
Col adding

```
def tableColadd():
```

```
    aq = ("alter table employee add (cname varchar2(10) not null)")
```

Q. Write a python program which will remove a table employee and student from oracle databases.

drop table

```
def tabledrop():
```

```
    aq = "drop table student"
```

Day 66

2) DML (Data Manipulation Language) Queries

→ The purpose of DML queries is that "to insert records, delete records and update records of any table".

→ DML (Data Manipulation Language) Queries are classified into 3 types

They are:-

i) insert

ii) delete

iii) update

→ After performing any DML operation through python program, we must commit the database by using commit() and to undo the operation, we do roll back by using rollback().

⇒ commit() and rollback() are present in connection object.

1) insert

→ This query is used for inserting record in table

syntax

```
SQL > insert into table-name values (val1, for col1, val2 for col 2  
... val-n for col-n)
```

example

```
SQL > insert into employee values (20, 'TR', 1.9, 'numpy');
```

2) delete

→ This query is used for deleting a record from table.

syntax: SQL > delete from table-name;

example) SQL > delete from employee; # Delete records of employee table

Syntax ②

SQL > delete from table-name where Cond list;

example

SQL > delete from employee where eno = 10; # Delete particular record of employee table.

3) update:

→ This Query is used for updating a record in a table.

Syntax ① :-

SQL > update table-name set Col1 = val1, Col2 = val2...
(OR) Col-n = val-n;

Syntax ② :-

SQL > update table-name set Col = val1, Col2 = val2...
Col-n = val-n where
Cond List

Write a python program which will accept employee value of a keyboard and insert them in employee table.

→ import cx_oracle

def recordinsert():

try:

con = cx_oracle.connect("system/manager@localhost/x")

cur = con.cursor()

iq = "insert into employee values (400, 'sagatika', 2.4, 'SE')"

cur.execute(iq)

con.commit()

print("Employee inserted successfully in Employee table")

except cx_oracle.DatabaseError as db:

print("problem in oracle DB", db)

main program

record insert

ex②. dynamically. from keyboard.

cursor take same value

accept the emp details from KBD

empno = int(input("Enter employee Number:"))

empname = input("Enter employee Name:")

empsal = float(input("Enter employee salary:"))

empdsig = input("Enter employee designation:")

prepare the Query with dynamic values.

iq = "insert into employee values (%d, '%s', %f, '%s')"

cur.execute(iq % (empno, empname, empsal, empdsig))

OR cur.execute("insert into employee values (%d, '%s', %f, '%s')"
% (empno, empname, empsal, empdsig))

con.commit

↓ Baaki sab same,

ex③

ex2 के सैट लिखना है।
while (True) के अंदर

while (True) के अंदर

→ print("{Employee inserted successfully in Employee table".format
(cur.execute)
print("-----")

ch = input("Do u want insert another record (yes/no):")

if (ch.lower() == "no"):

break

→ ex① का लिखना है।

except ValueError:

print("Don't enter Alphanumeric, and symbols for
empno, salary.")

Record Delete from employee

import cx_oracle

ex④. def deletereord():

try:

con = cx_oracle.connect("system/manager@129.0.0.1/x")

cur = con.cursor()

empno = int(input("Enter employee number for deleting record.

cur.execute("delete from employee where empno = %d" % empno)

con.commit()

if (cur.rowcount > 0):

print("{Employee record deleted".format(cur.rowcount)

else:

print("{emp Number does not exists".format(empno))

बाकी सब
record में
insert है

ex(1)

while true & Andar sab
else tak same

Ch=input("Do you want to ~~create~~ delete another record (yes/no):")
if (Ch.lower() == "no"):
break

↓ बाकी सब record insert ex(3) में है।

Update record

import cx_Oracle
def update_record():

try:

con = cx_Oracle.connect("system/manager @ 127.0.0.1 / xe")

cur = con.cursor()

empno = int(input("Enter Employee number for updating record:"))

cname = input("Enter new company name of employee:")

cur.execute("update employee set sal = sal + sal * (20/100)
, cname = '%s' where eno = %d

%, (cname, empno).

con.commit()

if (cur.rowcount > 0):

print("{ } employee record updated --- verify? format
(cur.rowcount))

else:

print("Employee number does not exist")

except: बाकी सब वही.

def del

while (True) के अंदर।

Accept employee details

empno = int(input("Enter employee number for updating Sal and
Comp name:"))

empsal = float(input("Enter employee new salary:"))

cname = input("Enter employee new Comp Name:")

Query with Dynamic values

uq = "update employee set sal = %.F, Comp name = '%s', where eno = %d"

cur.execute(uq % (empsal, cname, empno))

con.commit()

if (cur.rowcount

→ as if 0.

⊙


```
ch=input("Do you want to update another record (yes/no):")
```

```
if (ch.lower()=="no"):
```

```
    print("Thanks for using this program")
```

```
    break
```

बाकी सब वीH

Day 67

3) DRL (Data Retrieval Language) Queries

→ DRL (Data Retrieval Language) Queries are used for reading the records from table

→ To read the records from table, we use "select"

→ In other words "select" comes under DRL Query.

Syntax 1):

```
SQL > select col1, col2, .... Col-n from <table-name>
```

Syntax 2):

```
SQL > select col1, col2, .... Col-n from <table-name> where Cond list
```

Syntax 3):

```
SQL > select * from <table-name>
```

Syntax 4):

```
SQL > select * from <table-name> where Cond list
```

→ Once the select Query executed, all records are present in the object of cursor in python.

→ To get the records from cursor object, we have 3 functions

They are

i) fetchone()

ii) fetchmany (no. of records)

iii) fetchall()

i) fetchone()

→ This function is used for obtaining one record at a time, where cursor object pointing and it returns either tuple (if records exist) or None (if records does not exist).

1) fetchmany (no. of records)

→ `fetchmany (no. of records)` is used for obtaining specified number of records.

- Case 1) a) if specified number of records $= 0$ then this function obtains all records (in the case of `cx_oracle` module python 3.8, 3.9 & 3.10 only).
- ~~Case 1)~~ b) if specified number of records $= 0$ then this function select no records (in the case of `oracledb` -- python 3.11, 3.12 version module)
- Case 2): if specified number of records > 0 and specified number of records $\leq$ total number of records then this function gives specified number of records ~~this function obtain all records~~ in the case of both `cx_oracle` & `oracledb`.
- Case 3): if specified number of records $>$ total number of records then this function obtains all records in the case of both `cx_oracle` & `oracledb`.
- Case 4) if specified number of records < 0 then this function never gives any records.
- Case 5) if we don't specify specified number of records then this function obtains all records.

11) fetchall()

→ `fetchall` is used for obtaining all the records from cursor object in the form of tuple or list.

example). Consider the following stmt:-
 cur.execute("select * from employee")

cur. -->

ENO	ENAME	SAL	CNAME
100	Pragya	1.32	PST
300	Anif	1.57	infosys
500	Afri	1.98	wipro
600	Aman	1.99	HCL

Meta Data

Records area

program for reading the records from employee table - fetchone()

ex(1). import cx_oracle
 def getrecords():
 try:
 con = cx_oracle.connect("system/manager@127.0.0.1/xepdb1")
 cur = con.cursor()
 cur.execute("select * from employee")
 print("-----")
 while(True):
 record = cur.fetchone()
 if (record != None):
 for val in record:
 print("{} ".format(val), end="")
 print()
 else:
 print("-----")
 break
 except cx_oracle.DatabaseError as db:
 print("problem in oracle DB:", db)

main program
 getrecords()

ex(2). iskebaad records = cur.fetchmany(4)
 print("-----")
 for record in records:
 for val in record:
 print("{} ".format(val), end="")
 print()
 print("-----")

Back sab as 4

ex(3)

fetchall()

program for obtaining Col names along with records.

ex1) - Oracle table Col records - 1

import cx_Oracle

def getrecords():

try:

```
cm = cx_Oracle.connect("system/manager@127.0.0.1/xs")
```

```
cur = cm.cursor()
```

```
cur.execute("select * from employee")
```

```
print(" - - - - - ")
```

Code obtaining Col names.

```
colinfo = cur.description
```

```
for col in colinfo:
```

```
    print(col[0], end = "\t")
```

```
print()
```

```
print(" - - - - - ")
```

Code obtaining records.

```
records = cur.fetchall()
```

```
for record in records:
```

```
    for val in record:
```

```
        print("%s" % format(val), end = "\t")
```

```
    print()
```

Beaki's sub same

ex2) & for col in cur.description:

```
    print(col[0], end = "\t")
```

Beaki's sub same

program for accepting any table name and display records along with column names

Oracle table display

ex1)

accept the table name from keyboard.

```
print(" - - - - - ")
```

```
tname = input("Enter table name :")
```

```
cur.execute("select * from %s" % tname)
```

```
print(" - - - - - ")
```

Beaki's sub same

except) - print(" '%s' table does not exist")

MySQL Testing.

```
import mysql.connector  
print(mysql.connector.__version__)
```

Program for demonstrating connection from MySQL

ex①

```
import mysql.connector  
con = mysql.connector.connect(host = "localhost",  
                             user = "root", passwd = "root")  
print("\n Python program obtains connection from MySQL")  
print("Type of con =", type(con))
```

ex②) localhost # "127.0.0.1" भी लिख सकते हैं.

Program for demonstrating creating a cursor in MySQL

```
cur = con.cursor()  
print("\n Python program created a cursor object")  
print("Type of cur =", type(cur))  
=
```

Day 68

* Communication between Python program and MySQL

→ To develop Python and MySQL Communication (PDBC) Application, we need the following pre-requisites.

- 1) Install Python software
- 2) Install MySQL Data Base software
- 3) Install corresponding Data Base module for Python programmers

(Install MySQL-connector

MySQL connector -- Python)

→ After installing the above softwares, we develop the Application, which establish the communication Python software and MySQL Database software. To doing this we need to learn the following steps.

Step ①:- Import mysql.connector and other modules if required

→ When python programmer wants to communicate with mysql Database then python programmer must import mysql.connector

e.g). import mysql.connector

Step ②: Every python program must obtain the connection from mysql Database.

→ After importing either mysql.connector module, python program must get the connection from mysql Database

→ To get the connection from mysql Database, we must use connect(), which is present in mysql.connector

syntax:
varname = mysql.connector.connect(host="DNS/IpAddress",
user="user name of mysql Database",
passwd="password of mysql Database")

⇒ Here varname
↓
represents connection object of type <class, mysql.connector.connection>

⇒ Here mysql.connector
↓
represents name of predefined third party module

⇒ Here connect()
↓
it is one of the pre-defined function used for obtaining the connection from mysql.connector database.

⇒ user
↓
represents ~~name~~ user name of mysql Database

⇒ passwd
↓
represents password of mysql Database

⇒ DNS
↓
DNS stands for Domain naming system/service.
DNS represents name of the physical machines where Database software resides.

- Default DNS of every computer is "localhost"

⇒ IP Address

- ↓
- IP Address stands for Internet Protocol Address
- IP Address represents physical Address of the machine where Database software resides
- Default IP Address of every computer is 127.0.0.1 (Loopback Address)

→ if we write or specify any part of connection details as wrong then we get exception called "mysql.connector.Database Error"

Step ③ :- Every python program must create an object of cursor

→ The purpose of creating an object of cursor is that "to carry the Query from python program to Oracle Database and brings the result from Oracle Database and handover to python program".

→ To create an object of cursor, we use a pre-defined function called `cursor()` which is present in connection object.

syntax : $\text{Varname} = \text{connobj.cursor}()$
↓
Varname is an object of <class, `oracledb.cursor`>

Step ④ :- Every python must design the Query, place the Query in cursor object and send for execute in Database

→ A Query is a request/question to the Database software for performing any database operation.

→ To execute the Query from python program in Oracle Database, we use a pre-defined function `execute()`, which is present in cursor object.

syntax : $\text{curobj.execute}(\text{"Query"})$
↓
Can either DDL, DML, DCL

program for creating a database.

ex ① import mysql.connector
def database create():
try:
con = mysql.connector.connect(host="127.0.0.1",
user="root",
passwd="root")
cur = con.cursor()
dc = "create database batch 6pm"
cur.execute(dc)
print("database created successfully - verify")
except mysql.connector.DatabaseError as db:
print("problem in mysql DB", db)

main program
database create()
=

ex ② # mysql table create
password of root
database = "batch 6pm"
+ c = "create table employee (eno int primary key,
ename varchar(10) not null,
sal real not null, dsq varchar(10)
not null)"

database Employee table created successfully.

ex ③ # mysql table Add Column Name.
def table create()
aq = "alter table student add (colname varchar(10))"

ex ④ # mysql drop table
dq = "drop table student"

Q. write a python program which will insert a record in employee table
by accepting employee value from keyboard
⇒ mysql record insert

while True
try: cur

mysql record delete (2) Code toh baar firse dekh leme
def record delete()

mysql record update

mysql record select

← just at is
oracle